LangNation Cloud Platform

Simple Project Study Guide — Milestones 1–6

What I built • why each AWS service is used • how everything connects • how I troubleshoot it

Author: SERAJ AHMAD SIDDIQUI
AWS Cloud Computing with AI Bootcamp
Current scope: Milestones 1–6 + automated sandbox recovery
Region: us-east-1

Purpose: This is a learning and revision document. Read it from top to bottom when you want to understand the complete project, or jump to one milestone when you need to review a specific AWS topic.

Contents

1. Project in one page
2. Shared tools and control model
3. Milestone 1 — Foundation & Networking
4. Milestone 2 — Docker, ECR & ECS Fargate
5. Milestone 3 — ALB, Multi-AZ & Auto Scaling
6. Milestone 4 — Automated Deployment
7. Milestone 5 — Database & Storage
8. Milestone 6 — Edge, DNS, Security & Authentication
9. OP01 — Automated Morning Sandbox Recovery
10. Complete end-to-end flows
11. Troubleshooting by symptom
12. AWS service glossary
13. Interview / presentation revision
14. Planned next phases

Important: Milestones 7–10 are planned only. This guide does not describe them as completed functionality.

1. Project in One Page

LangNation Cloud Platform is a Terraform-managed AWS learning platform. I built it step by step so that each milestone adds one new capability without throwing away the previous architecture.

```
User
  ↓
Route 53
  ↓
CloudFront + ACM + AWS WAF
  ↓
Application Load Balancer
  ↓
Target Group
  ↓
Private ECS Fargate tasks across 2 AZs
  ↓
RDS MySQL / Amazon S3

Authentication:
User → Cognito → Access Token → /protected → JWT verification

Deployment:
Git push → GitHub Actions → OIDC → ECR → ECS rolling deployment

Infrastructure:
Terraform code → HCP Terraform → AWS

Recovery:
Activate training sandbox manually
  ↓
HCP Terraform rebuild
  ↓
GitHub Actions restores application
  ↓
Route 53 points to recreated CloudFront
  ↓
/health verification
```

What problem does the project solve?

- Run an application in AWS without exposing the application containers or database directly to the internet.
- Keep the application available across two Availability Zones.
- Automatically build and deploy application changes from GitHub.
- Store relational data and private files securely.
- Add HTTPS, WAF protection and authentication.
- Rebuild the environment after the training sandbox deletes AWS resources.

The most important design idea

Mental model: The AWS training environment is disposable, but the definition of the platform is durable. Terraform, Git, HCP Terraform state and persistent DNS allow the environment to be recreated instead of manually rebuilt.

2. Shared Tools and Control Model

Tool / Service	What I use it for	Simple explanation
VS Code	Edit application, Terraform and workflow files	My local working environment.
Git + GitHub	Source control	Stores history and lets me push changes safely.
Terraform	Infrastructure as Code	Describes AWS resources in code.

Tool / Service	What I use it for	Simple explanation
HCP Terraform	Authoritative plan, apply and remote state	The trusted place that remembers Terraform state and applies infrastructure.
GitHub Actions	CI/CD and recovery automation	Runs deployment and recovery workflows automatically.
AWS CLI	Validation and troubleshooting	Lets me inspect AWS from the terminal.
Docker	Package the Node.js app	Creates the same portable application image for local and AWS runtime.

Who owns what?

Owner	Owns	Why
HCP Terraform	Infrastructure and Terraform state	Avoids local state confusion and recreates infrastructure consistently.
GitHub Actions	Application image and live ECS task-definition revision	Application releases change more often than infrastructure.
ECS Auto Scaling	Live desired task count	Scaling must react dynamically to CPU, not wait for Terraform.
Route 53 recovery step	Public DNS destination	CloudFront domain can change after sandbox rebuild.

Key rule: Do not let Terraform, CI/CD and Auto Scaling fight over the same runtime value. That is why the ECS service ignores live changes to desired_count and task_definition.

Normal Terraform workflow

```
Local workstation
  ↓
terraform init
terraform fmt -recursive
terraform validate
  ↓
Git / GitHub
  ↓
HCP Terraform
  ↓
Plan → Review → Apply
  ↓
AWS
```

Local Terraform is mainly used for init, formatting and validation. HCP Terraform is the authoritative source for plan, apply, state and run history.

3. Milestone 1 — Foundation & Networking

Goal

Build the network foundation first, so later compute and database services have a secure place to run.

AWS services used and why

AWS service	Why I used it	How it connects
Amazon VPC	Creates my private AWS network.	Contains all public, app and DB subnets.
Public subnets	Hold internet-facing resources.	ALB and NAT Gateways live here.
Private app subnets	Keep ECS tasks private.	Tasks use NAT for outbound access.
Private DB subnets	Isolate the database.	No default internet route.
Internet Gateway	Gives public subnets an internet path.	Public route table sends 0.0.0.0/0 to IGW.
NAT Gateway	Lets private app workloads reach outward without becoming public.	One NAT per AZ; app subnet uses same-AZ NAT.
Route tables	Control where network traffic goes.	Separate public, app and DB routing.
Security Groups	Control allowed traffic between layers.	ALB → ECS → DB.

Network layout

```
VPC 10.0.0.0/16

us-east-1a                          us-east-1b
-----------------------------      -----------------------------
Public  10.0.1.0/24                Public  10.0.2.0/24
App     10.0.11.0/24               App     10.0.12.0/24
DB      10.0.21.0/24               DB      10.0.22.0/24

Public RT: 0.0.0.0/0 → Internet Gateway
App RT 1: 0.0.0.0/0 → NAT Gateway 1
App RT 2: 0.0.0.0/0 → NAT Gateway 2
DB RTs: local routes only
```

Step-by-step

1. Create the VPC with CIDR 10.0.0.0/16.
2. Create six /24 subnets across us-east-1a and us-east-1b.
3. Attach an Internet Gateway.
4. Create a public route table and route 0.0.0.0/0 to the Internet Gateway.
5. Create two Elastic IPs and two NAT Gateways, one in each public subnet.
6. Route each private application subnet through the NAT Gateway in the same AZ.
7. Keep the private database route tables local-only.
8. Create layered security groups so internal access follows ALB → ECS → DB.
9. Expose module outputs and connect modules through Terraform rather than hardcoded AWS IDs.
10. Validate locally, then use HCP Terraform for authoritative plan/apply.

How traffic works

```
Internet → Internet Gateway → public subnet resources

Private ECS task → private app route table → same-AZ NAT → Internet

ECS → DB Security Group → RDS on TCP 3306

Database subnet → no direct Internet route
```

Errors I met and how I fixed them

Problem	Why it happened	Fix / lesson
Terraform: Unsupported block type	A variable/configuration block had a missing closing brace.	Correct the braces, then run terraform validate.
Module not installed	A new Terraform module had been added.	Run terraform init again.
Local plan wanted to recreate existing resources	Local state was not the authoritative HCP state.	Use local init/fmt/validate; use HCP Terraform for plan/apply/state.
Sandbox removed resources overnight	Training environment is disposable.	Treat this as a reproducibility test and rebuild from Terraform.

Remember M1: A subnet is not public or private because of its name. Routing determines its behavior.

4. Milestone 2 — Docker, ECR & ECS Fargate

Goal

Turn the application into a container and run it as a private managed workload on AWS.

AWS services and technologies used

Service / technology	Why I used it	How it connects
Node.js / Express	Small controlled capstone API.	Provides / and /health on port 3000.
Docker	Packages the application consistently.	Image is pushed to ECR.
Amazon ECR	Private Docker registry.	ECS task definition references the image.
Amazon ECS	Orchestrates containers.	Service keeps the requested number of tasks running.
AWS Fargate	Runs containers without managing EC2 servers.	Tasks run inside private app subnets.
IAM execution role	Allows ECS to start the task.	Pulls from ECR and sends logs.
CloudWatch Logs	Central application logs.	Fargate streams container output here.

Application container

```
Source code
   ↓
Dockerfile
   ↓
Docker image
   ↓
Amazon ECR
   ↓
ECS Task Definition
   ↓
ECS Service
   ↓
Fargate task in private app subnet
   ↓
CloudWatch Logs
```

- Base image: node:22-alpine
- Port: 3000
- CPU: 256
- Memory: 512 MiB
- Network mode: awsvpc
- Public IP: disabled
- Container runs as non-root user node
- CloudWatch log retention: 7 days

Important ECS concepts

Concept	Simple meaning
Cluster	Logical home for ECS services and tasks.
Task Definition	Blueprint: image, CPU, memory, ports, environment, logging.
Task	One running copy of the blueprint.
Service	Controller that keeps the requested number of tasks running.
Fargate	AWS-managed compute that actually runs the task.

Errors I met and how I fixed them

Problem	Cause	Fix / lesson
Docker Linux engine would not start	WSL 2 was not available/configured.	Run wsl --install, configure WSL 2/Ubuntu, then restart Docker Desktop.
ECR repository returned after rebuild but image was gone	Terraform recreates infrastructure, not container artifacts.	Rebuild and repush the image; later automate this with GitHub Actions.
ECS task health showed Unknown	No ALB target-group health check existed yet.	Keep /health in the app; connect it to ALB health checks in M3.

Remember M2: Terraform can recreate the ECR repository, but it cannot recreate the Docker image stored inside it. Infrastructure recovery and application artifact recovery are different jobs.

5. Milestone 3 — ALB, Multi-AZ & Auto Scaling

Goal

Make the private application reachable through a controlled public entry point, distribute it across two AZs, and scale task count automatically.

Services used and why

AWS service	Why I used it	How it connects
Application Load Balancer	Public entry point and traffic distribution.	Forwards requests to the target group.
Target Group	Tracks healthy ECS targets.	Uses target type ip and port 3000.
Health Check	Stops traffic going to unhealthy tasks.	GET /health must return HTTP 200.
ECS Service	Maintains two baseline tasks.	Tasks are registered in the target group.
Application Auto Scaling	Changes desired task count automatically.	CPU target tracking scales 2–4 tasks.

Request path at this milestone

```
Internet
  ↓
ALB :80
  ↓
Target Group :3000
  ↓
Healthy ECS Fargate task
  ↓
Node.js application
```

Availability and scaling

- Baseline: 2 running ECS tasks.
- One task is placed in each Availability Zone.
- Auto Scaling minimum: 2 tasks.
- Auto Scaling maximum: 4 tasks.
- Normal target: 50% average CPU.
- Scale-out cooldown: 60 seconds.
- Scale-in cooldown: 300 seconds.

For a controlled test, the CPU target was temporarily reduced to 1%. Generated load caused ECS to scale automatically from 2 to 4 tasks. The normal 50% target was then restored.

Why ignore desired_count in Terraform?

```
Terraform
  → defines service + scaling configuration

Application Auto Scaling
  → owns live desired task count
```

If Terraform tried to force desired_count back to a fixed value after Auto Scaling changed it, the two systems would fight. The ECS service therefore ignores live desired_count changes.

Errors I met and how I fixed them

Problem	Cause	Fix / lesson
Terraform initialized in an empty directory	Terminal was at repo root.	cd into terraform/environments/dev before Terraform commands.
New load-balancing module missing from diff	Folder was untracked.	Use git status --short, not only git diff.
Trailing whitespace found	Invisible spaces in staged files.	Run git diff --cached --check before commit.
ignore_changes on wrong resource	desired_count belongs to ECS Service, not Task Definition.	Move lifecycle rule to the ECS Service.
ECS network block accidentally removed	Refactor changed more than intended.	Review staged diff; restore private subnets, ECS SG and no-public-IP settings.

Remember M3: High availability is not just 'two resources'. They must be placed across separate failure domains and traffic must be health-aware.

6. Milestone 4 — Automated Deployment

Goal

Replace manual Docker/ECR/ECS deployment with a repeatable GitHub Actions pipeline that uses short-lived AWS credentials.

Services used and why

Service	Why I used it	How it connects
GitHub Actions	Runs deployment automatically after application changes.	Builds image and updates ECS.
GitHub OIDC	Avoids long-lived AWS access keys.	GitHub identity is exchanged for AWS role credentials.
AWS STS	Issues short-lived credentials.	Allows the workflow to assume the deployment role.
IAM deployment role	Least-privilege permissions.	Allows only needed ECR/ECS/PassRole operations.
Amazon ECR	Stores immutable Git-SHA images.	ECS task definitions point to the new image.
Amazon ECS	Performs rolling deployment.	Replaces tasks gradually with the new revision.

Deployment flow

```
Developer changes application
  ↓
git push main
  ↓
GitHub Actions
  ↓
OIDC token
  ↓
AWS STS / IAM deployment role
  ↓
Docker build
  ↓
Image tag = Git commit SHA
  ↓
Amazon ECR
  ↓
New ECS Task Definition revision
  ↓
Rolling ECS deployment
  ↓
Wait for stable service
  ↓
Verify exact image SHA + task counts + public /health
```

Why Git-SHA image tags?

A Git SHA creates a direct link between source code, Docker image, task-definition revision and the running deployment. I can identify exactly which commit is running instead of relying only on a moving tag such as dev.

Why OIDC?

Security: GitHub does not store permanent AWS deployment keys. GitHub proves its identity, AWS STS issues temporary credentials, and IAM limits what the workflow can do.

Ownership after CI/CD

```
Terraform           → infrastructure
GitHub Actions      → live task_definition
Auto Scaling        → live desired_count
```

The ECS Service lifecycle ignores task_definition and desired_count so Terraform does not roll back a newer CI/CD deployment or undo a scaling decision.

Errors I met and how I fixed them

Problem	Cause	Fix / lesson
sts:AssumeRoleWithWebIdentity denied	OIDC trust policy did not match GitHub's actual immutable subject.	Use immutable owner/repository IDs, main branch and correct STS audience.
Terraform could revert a GitHub deployment	Both systems could control task_definition.	Ignore task_definition in ECS Service lifecycle; CI/CD owns live revisions.
Deployment action reported failure but ECS later became healthy	Transient rollout state was interpreted as failure.	Independently run aws ecs wait services-stable, verify image SHA, counts and public health.

Remember M4: CI/CD is not only a YAML file. Identity, authorization, ownership and final-state validation are part of the deployment design.

7. Milestone 5 — Database & Storage

Goal

Add persistent relational data and private object storage without exposing either service publicly.

Services used and why

AWS service	Why I used it	How it connects
Amazon RDS MySQL	Relational application data.	ECS reaches it privately on TCP 3306.
RDS DB Subnet Group	Places RDS in the two private DB subnets.	Uses us-east-1a and us-east-1b database subnets.
RDS Multi-AZ	Improves database availability.	AWS maintains a synchronous standby.
AWS Secrets Manager	Keeps the RDS master password out of code.	RDS manages the master credential.
Amazon S3	Private object storage and future RAG content.	Application accesses objects over HTTPS.
S3 Versioning	Keeps older object versions.	Protects against accidental overwrite/delete.
S3 Lifecycle	Controls storage growth.	Cleans older noncurrent versions and incomplete multipart uploads.

RDS configuration

Setting	Value / reason
Engine	MySQL; deployed version observed as 8.4.11.
Instance	db.t3.micro for the development environment.
Storage	20 GiB gp3, autoscaling up to 100 GiB.
Encryption	Enabled at rest.
Public access	Disabled.
Multi-AZ	Enabled.
Backup retention	7 days.
Database name	langnation.
Master password	Managed by AWS Secrets Manager.

S3 security

- Block Public Access: all controls enabled.
- Object ownership: BucketOwnerEnforced.
- Versioning: enabled.
- Encryption: SSE-S3 / AES256.
- Bucket policy: HTTPS-only.
- Lifecycle: remove old noncurrent versions after 30 days and abort incomplete multipart uploads after 7 days.

Data flow

```
ECS task
   ├─→ DB Security Group → RDS MySQL :3306
   │
   └─→ HTTPS → private Amazon S3 bucket

RDS:
Private DB subnets
No public access
Multi-AZ
Encrypted
Credentials managed by Secrets Manager
```

Errors I met and how I fixed them

Problem	Cause	Fix / lesson
RDS engine/version discovery APIs denied	Training sandbox blocked DescribeDBEngineVersions / orderable options.	Do not pin an explicit engine version; let AWS select a supported version. Deployment succeeded.
Module not installed	New database/storage module added.	Run terraform init.
Terminal output accidentally pasted as PowerShell commands	Command and output text were copied together.	Copy only the actual command when troubleshooting.

Remember M5: Having a DNS endpoint does not make a database public. Public/private behavior depends on subnet placement, routes, security groups and the RDS publicly_accessible setting.

8. Milestone 6 — Edge, DNS, Security & Authentication

Goal

Put a secure public edge in front of the ALB, keep DNS persistent outside the disposable sandbox, block direct ALB bypass and add user authentication.

Services used and why

AWS service	Why I used it	How it connects
Amazon Route 53	Stable public DNS.	Persistent personal AWS zone points cloud.namienterprises.in to CloudFront.

AWS service	Why I used it	How it connects
AWS Certificate Manager	TLS certificate for HTTPS.	CloudFront uses the certificate; DNS validation remains persistent.
Amazon CloudFront	Secure public front door.	Terminates viewer HTTPS and forwards allowed traffic to ALB.
AWS WAF	Filters malicious/suspicious requests.	Attached to CloudFront.
CloudFront managed prefix list	Prevents users bypassing CloudFront.	ALB port 80 accepts only CloudFront origin-facing addresses.
Amazon Cognito	User authentication and access tokens.	Application verifies Cognito JWT access tokens.
aws-jwt-verify	Validates tokens inside Node.js.	Protects /protected while /health stays public.

Current public request path

```
User
  ↓
Route 53
  ↓
CloudFront
  ├─ ACM certificate
  └─ AWS WAF
  ↓ HTTP :80 to origin
Application Load Balancer
  ↓
Target Group
  ↓
Private ECS task
  ↓
Node.js application
```

TLS boundary: HTTPS is from the user to CloudFront. CloudFront currently reaches the ALB over HTTP port 80. This project does not claim end-to-end TLS to the origin.

Why persistent DNS?

The training sandbox can delete CloudFront and ACM, so those resource IDs and endpoints can change. Route 53 stays in a persistent personal AWS account. This creates the principle: Persistent DNS + disposable infrastructure.

WAF and direct-ALB protection

- AWSManagedRulesCommonRuleSet is attached.
- Per-IP rate rule limit: 1000 requests.
- Controlled XSS-style request returned HTTP 403.
- Normal /health request returned HTTP 200.
- ALB ingress on port 80 is restricted to the AWS-managed CloudFront origin-facing prefix list.
- Direct ALB request is blocked/times out, so users cannot bypass WAF.

Cognito authentication

```
User signs in
  ↓
Amazon Cognito
  ↓
Access token
  ↓
GET /protected
Authorization: Bearer <token>
  ↓
Node.js uses aws-jwt-verify
  ↓
Valid token → HTTP 200
Missing / invalid token → HTTP 401
```

- Email-based sign-in and verification.
- Public browser client: no client secret.
- Access token: 60 minutes.
- ID token: 60 minutes.
- Refresh token: 30 days.
- prevent_user_existence_errors enabled.

Errors I met and how I fixed them

Problem	Cause	Fix / lesson
DNS delegation looked inconsistent	DNS propagation takes time after nameserver changes.	Wait for propagation before repeatedly changing configuration.
ACM ARN changed after sandbox cleanup	Certificate is disposable with the sandbox.	Use Terraform dependencies/outputs; do not hardcode disposable ARNs.
ECS deploy action false-failed	Transient deployment state.	Use independent service-stability, image-SHA, task-count and CloudFront health checks.
PowerShell AWS CLI shorthand parsing failed	Compound comma-separated CLI arguments were parsed unexpectedly.	Quote compound values carefully or store the full argument in a variable.

Remember M6: The public URL stays stable even when the sandbox infrastructure changes underneath it. DNS is the stable pointer; CloudFront/ALB/ECS are rebuildable runtime components.

9. OP01 — Automated Morning Sandbox Recovery

Goal

Recover the disposable AWS environment after the training platform has removed resources, without manually recreating the infrastructure in the AWS Console.

Manual prerequisite: The neue-fische training sandbox itself must be activated manually first. The automation cannot start the educational lab session. Once the sandbox has an active AWS session, the recovery workflow can take over.

Why recovery needs more than Terraform

```
Night cleanup can remove:
  1. AWS infrastructure
  2. Docker images stored in recreated ECR
  3. CloudFront endpoint identity

Terraform can rebuild infrastructure,
but GitHub Actions must restore the application artifact,
and Route 53 must follow the new CloudFront endpoint.
```

Recovery flow

```
Manual: activate training sandbox
  ↓
07:37 primary workflow (or manual workflow_dispatch)
  ↓
HCP Terraform API
  ↓
Dedicated recovery run with auto-apply
  ↓
Infrastructure recreated
  ↓
Reusable deploy.yml
  ↓
Docker image rebuilt + pushed to ECR
  ↓
ECS service updated and validated
  ↓
Read new CloudFront domain from HCP Terraform output
  ↓
GitHub assumes restricted personal-AWS Route 53 role
  ↓
UPSERT cloud.namienterprises.in
  ↓
Wait for DNS INSYNC
  ↓
Verify HTTPS /health

08:17 fallback:
  healthy → skip
  unhealthy → run recovery
```

Why two schedules?

The 07:37 run is the primary recovery attempt. The 08:17 fallback first checks the public health endpoint. If the site is already healthy, it skips the duplicate rebuild. If not, it retries recovery.

Why per-run auto-apply?

Normal HCP Terraform development keeps global auto-apply OFF so infrastructure changes can be reviewed. Only the known recovery run requests auto-apply. This keeps normal development controlled while allowing repeatable recovery.

Important failure we discovered

Symptom	Root cause	Resolution

Symptom	Root cause	Resolution
HCP Terraform showed many explicit-deny errors for EC2/IAM/WAF/CloudFront/Cognito/ACM reads.	The training sandbox was inactive, so the AWS session was not available.	Activate the training sandbox manually, then rerun recovery. After activation, the full rebuild/deploy/DNS/health workflow passed.

What a green recovery run means

- Terraform infrastructure is ready.
- Docker image is rebuilt and pushed.
- ECS deployment is stable.
- Running image matches the intended Git SHA.
- Desired and running task counts match; pending is zero.
- CloudFront destination was discovered.
- Route 53 was synchronized.
- Public HTTPS /health succeeded.

Remember OP01: The correct description is: manual sandbox activation → automated infrastructure rebuild → automated application restore → automated DNS synchronization → automated health validation.

10. Complete End-to-End Flows

10.1 What happens when a user opens the public URL?

1. The browser requests cloud.namienterprises.in.
2. Route 53 resolves the hostname to CloudFront.
3. CloudFront terminates HTTPS using the ACM certificate.
4. AWS WAF evaluates the request.
5. If allowed, CloudFront sends the request to the ALB over HTTP port 80.
6. The ALB forwards the request to the target group.
7. The target group selects a healthy private ECS task on port 3000.
8. The Node.js application returns the response.

10.2 What happens when a protected endpoint is called?

1. The user authenticates with Cognito.
2. Cognito returns an access token.
3. The client sends Authorization: Bearer <token> to /protected.
4. The request still travels through Route 53 → CloudFront/WAF → ALB → ECS.
5. The Node.js application validates signature, user pool, client ID, token use and expiration.
6. Valid token returns HTTP 200; missing/invalid token returns HTTP 401.

10.3 What happens when I push application code?

1. A relevant push to main triggers deploy.yml.
2. GitHub Actions requests an OIDC identity token.
3. AWS STS issues temporary credentials for the restricted deployment role.

4. The workflow builds the Docker image and tags it with the Git SHA.
5. The image is pushed to ECR.
6. A new ECS task-definition revision is registered.
7. ECS performs a rolling deployment.
8. The workflow independently waits for service stability.
9. It verifies the running image SHA, task counts and public /health through CloudFront.

10.4 What happens when Terraform changes?

1. Edit code under terraform/environments/dev or terraform/modules.
2. Run terraform init when needed, then fmt and validate locally.
3. Commit and push the change.
4. HCP Terraform creates the authoritative plan using remote state.
5. Review and apply the run.
6. HCP Terraform provisions or updates AWS resources.

10.5 What happens when the sandbox was cleaned?

1. Activate the training sandbox manually.
2. Run/wait for the recovery workflow.
3. HCP Terraform detects missing resources and rebuilds them.
4. GitHub Actions rebuilds the application image and deploys it.
5. The workflow reads the newly recreated CloudFront domain.
6. A restricted cross-account OIDC role updates persistent Route 53.
7. The workflow waits for DNS INSYNC and verifies the public HTTPS health endpoint.

11. Troubleshooting by Symptom

Symptom	Check first	Likely action
Terraform says Module not installed	Did you add/change a module?	Run terraform init.
Terraform initialized in empty directory	Current path	cd terraform/environments/dev.
HCP plan shows many explicit-deny read errors	Is the training sandbox active?	Activate sandbox, then rerun.
Local Terraform plan looks completely wrong	Are you relying on local state?	Use HCP Terraform as authoritative state/plan/apply.
Docker Desktop cannot start Linux containers	WSL 2 status	Install/configure WSL 2 and restart Docker.
ECR exists but image is missing after rebuild	Was the sandbox recreated?	Run application deployment to rebuild/push image.
ECS task keeps stopping	CloudWatch logs + task stopped reason	Fix app/startup/config; ECS will replace task.
Target is unhealthy	/health response, port 3000, SG path	Fix app health, target group or ALB→ECS access.
Direct ALB URL times out	Are you on M6 current architecture?	Expected. Use cloud.namienterprises.in through CloudFront.

Symptom	Check first	Likely action
Public endpoint returns 403	Was request intentionally suspicious?	Could be WAF working. Inspect WAF rule/matched request.
/protected returns 401	Token present, not expired, correct client/pool?	Get a valid Cognito access token and retry.
/protected returns 503	Cognito verifier env values available?	Check COGNITO_USER_POOL_ID / COGNITO_CLIENT_ID in task definition.
RDS connection fails	DB SG, port 3306, private placement, credentials	Verify ECS SG → DB SG rule and Secrets Manager-managed credential path.
DNS still shows old destination	Route 53 change INSYNC? resolver cache?	Wait for DNS propagation/TTL; verify CNAME and nameservers.
ACM certificate cannot be reused by ARN	Was sandbox recreated?	Do not hardcode ARN; let Terraform create/validate new certificate.
GitHub OIDC role assumption denied	aud/sub/role/repository/branch	Verify immutable GitHub subject and trust-policy conditions.
GitHub deploy action says failed but ECS looks healthy	Final ECS state	Use independent services-stable wait, image-SHA and health checks.
git diff --check reports trailing whitespace	Reported file/line	Remove trailing spaces or use explicit when a Markdown line break is intended.

My troubleshooting order

```
1. Read the exact error
2. Identify the layer:
   local tool / Terraform / IAM / network / ECS / application / DNS / auth
3. Check the simplest prerequisite first
4. Use logs and AWS state, not guesses
5. Change one thing
6. Re-run validation
7. Confirm the final end-to-end result
```

12. AWS Service Glossary — Simple Meanings

Term	Simple meaning
VPC	My isolated AWS network.
Subnet	A smaller network range inside the VPC.
Internet Gateway	Internet doorway for public subnet routing.
NAT Gateway	Lets private workloads go outward without accepting inbound internet traffic.
Route Table	Rules that decide where network packets go.
Security Group	Stateful firewall attached to AWS resources/network interfaces.
ECR	Private Docker image registry.
ECS	AWS container orchestration service.
Fargate	Serverless compute for ECS containers; no EC2 server management.

Term	Simple meaning
Task Definition	Blueprint for an ECS task.
ECS Service	Keeps a desired number of tasks running.
ALB	HTTP/HTTPS load balancer that distributes requests.
Target Group	Set of backend targets plus health checks.
Auto Scaling	Automatically changes the number of ECS tasks.
CloudWatch	Logs, metrics and operational visibility.
IAM	AWS identity and permission system.
OIDC	Identity federation method used by GitHub to get temporary AWS access.
STS	AWS service that issues temporary credentials.
RDS	Managed relational database service.
Secrets Manager	Secure place for managed credentials/secrets.
S3	Object/file storage.
Route 53	AWS DNS service.
ACM	AWS certificate service for TLS/HTTPS.
CloudFront	AWS edge/CDN service used here as the secure public front door.
WAF	Web Application Firewall.
Cognito	Managed user authentication service.
JWT	Signed token that proves authenticated access.
Terraform	Infrastructure as Code tool.
HCP Terraform	Remote Terraform state, plan and apply platform.

13. Interview / Presentation Revision

30-second project explanation

Suggested answer: I built a multi-AZ AWS platform for LangNation using Terraform and HCP Terraform. The Node.js application runs as private ECS Fargate tasks behind an ALB. CloudFront, ACM and WAF provide the public HTTPS edge, Cognito protects authenticated endpoints, RDS MySQL and S3 provide the data layer, and GitHub Actions deploys immutable Git-SHA container images using OIDC instead of static AWS keys. Because the training sandbox is regularly cleaned, I also built a recovery workflow that rebuilds the infrastructure after the sandbox is activated, restores the application, updates persistent Route 53 DNS and verifies the public health endpoint.

One-line explanation per milestone

Milestone	What I added	Main lesson

Milestone	What I added	Main lesson
M1	VPC, subnets, routing, NAT, security groups, Terraform/HCP	Build a secure network foundation first.
M2	Docker, ECR, ECS Fargate, CloudWatch	Infrastructure and application artifacts are separate concerns.
M3	ALB, target group, health checks, multi-AZ, Auto Scaling	Availability requires distribution + health-aware routing.
M4	GitHub Actions, OIDC, STS, immutable deployments	Automation needs clear identity and ownership.
M5	RDS MySQL, Secrets Manager, S3	Keep data private, encrypted and recoverable.
M6	Route 53, ACM, CloudFront, WAF, Cognito, JWT	Secure the public edge and make DNS stable.
OP01	HCP recovery + deploy + DNS sync + health validation	After manual sandbox activation, recovery can be automated end to end.

Questions I should be able to answer

- Why are ECS tasks private?
- What is the difference between an Internet Gateway and a NAT Gateway?
- Why does Fargate use target type ip?
- Why are two NAT Gateways used?
- Why does Terraform ignore ECS desired_count and task_definition?
- Why use OIDC instead of AWS access keys?
- Why use Git-SHA image tags?
- Why is RDS in private DB subnets?
- Why is Route 53 outside the training sandbox?
- Where does HTTPS terminate?
- Why can the ALB not be called directly anymore?
- How does Cognito protect /protected?
- What happens after nightly sandbox cleanup?

14. Planned Next Phases

Status: The following phases are planned. They are not yet completed in the current project state.

Milestone	Planned topic	Expected direction
M7	Serverless & Event-Driven Architecture	Likely event processing with services such as Lambda, EventBridge and SQS.
M8	RAG Knowledge Pipeline	Document ingestion, processing, embeddings and knowledge retrieval foundation.
M9	Lina AI Tutor / RAG Runtime	Use the knowledge pipeline to provide AI-assisted answers.
M10	Security, Monitoring, CI/CD Quality Gates, Resilience & Final Architecture	Final hardening, observability, resilience testing and project completion.